Mission 6: The Autonomous Warp State Machine & Deliberation Loop

Starship Operational Unit: USSE Noosphere Bridge Crew
Course: Conversational AI — Winter Semester 2026/27
Module: Week 6 — The Agentic Shift: System 1 vs. System 2 Thinking
Galactic Waypoint: Sector SEC-06-CAR: Eta Carinae & Homunculus Nebula (7,500 ly from Sol)
AURA Subsystem Repair: Subsystem 06: Autonomous Warp State Machine & Deliberation Loop (+7.1% -> 42.6% total integrity)
IDE & Tools Standard: Visual Studio Code + Antigravity (`agy`) | Python 3.11+ / PyTorch 2.4+ / Dataclasses & Enums (Zero External Framework Bloat)
Deliverable: `ps06_solution.ipynb`

---

1. Mission Briefing

Cadets, the USSE Noosphere has entered **Sector SEC-06-CAR**, navigating the treacherous and highly unstable stellar winds of the Eta Carinae binary system and its surrounding Homunculus Nebula. The intense radiation and gravitational shear forces have knocked out our primary navigation relays.

Worse, AURA's higher-order logic subroutines have collapsed under the data influx. She has reverted to **System 1 greedy reflex processing**—making rapid, impulsive decisions without thinking ahead, triggering infinite execution loops, and draining our remaining plasma reserves. If she continues on this trajectory, she will trap the ship in a permanent, infinite action-observation loop within the nebula.

Your mission is to upgrade AURA to **System 2 thinking** by engineering a deterministic, haltable **Autonomous Warp State Machine & Deliberation Loop**. You must restore her ability to reason, deliberate, and safely execute tool calls while enforcing strict step budgets and preventing infinite looping traps.

---

2. Four Squadron Role Directives & Deliverables

Divide your bridge crew into the following four roles. Each cadet is responsible for implementing their designated section within the final integrated `ps06_solution.ipynb` notebook.

Part 1: Systems Architect (Lead Engineer) — Native Python Agent State Machine Core

Objective: Design the core deterministic state machine that governs AURA's execution cycle, ensuring she transitions logically and cannot exceed a safe execution budget.

Tasks:

1. **State Machine Definition:** Define a formal state machine using Python `Enum`. Create the `AgentState` enum with the following exact states:

- `IDLE`, `PERCEIVING`, `DELIBERATING`, `ACTING`, `OBSERVING`, `REFLECTING`, `HALTED`

2. **Agent Dataclass:** Implement an `Agent` class using standard Python `@dataclass`.
3. **Step Transition Function:** Create the `step()` function which executes exactly one state transition according to a strict graph:
   - `IDLE` → `PERCEIVING`
   - `PERCEIVING` → `DELIBERATING`
   - `DELIBERATING` → `ACTING`
   - `ACTING` → `OBSERVING`
   - `OBSERVING` → `REFLECTING`
   - `REFLECTING` → `DELIBERATING` (Loop) OR `HALTED` (Terminal)
4. **Step Budget:** Enforce a strict step budget (Max Steps $N \leq 20$). If the step count exceeds N, the agent must forcibly transition to `HALTED`.

Part 2: Communications & Protocol Officer — Tool Protocol & Action Dispatcher

Objective: Standardize how AURA requests and executes external tools, validating all inputs before they interface with critical ship systems.

Tasks:

1. **Tool Definitions:** Define three specific tool functions required to survive Eta Carinae:
   - `navigate_warp_vector(x: float, y: float, z: float) -> str`
   - `adjust_plasma_shields(intensity: int) -> str`
   - `query_sensor_array(target: str) -> str`
2. **JSON-Schema Validator:** Implement a strict JSON-Schema tool contract validator for the action dispatcher. Before a tool is executed, validate the LLM's requested arguments against the schema.
3. **Error Boundary:** Wrap all tool executions in an error boundary. If a tool fails (e.g., `ValueError` due to negative shield intensity), catch the exception, format it as an observation (`"Tool Execution Error: <msg>"`), and inject it back into the agent's context gracefully so she can correct her mistake in the next deliberation cycle.

Part 3: Data & Payload Specialist — MDP Trajectory Logger & Empirical Return Evaluator

Objective: Record and evaluate AURA's performance as a Markov Decision Process (MDP) to ensure her actions are optimizing for long-term ship survival, not short-term rewards.

Tasks:

1. **Trajectory Logging:** Implement a logger that records state-action-reward trajectories over an episode:

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, ..., s_T)$$

2. **Discounted Return:** For a given completed trajectory, write a function to calculate the empirical discounted cumulative return G_0:

$$G_0 = \sum_{t=0}^{T} \gamma^t r_t$$

(Use a discount factor $\gamma = 0.95$)
3. **Transition Frequency:** Compute a transition frequency matrix representing the empirical probability $P(s' \mid s, a)$ of successfully navigating the turbulent nebula sectors based on a provided historical dataset of past jumps.

Part 4: Security & Ethics Sentinel — Cycle Detection & Halting Guardrail Sentinel (Ship Anomaly 06)

Objective: AURA's impulsive actions are causing her to repeat the same sequence of flawed actions infinitely. You must detect these cycles in real-time and trigger an emergency breaker.

Tasks:

1. **Tortoise and Hare Detection:** Implement **Floyd's Tortoise and Hare algorithm** (cycle detection) to monitor the agent's execution trajectory τ.
2. **State-Action Hashing:** Implement a state-hashing mechanism that hashes the current environmental state and AURA's planned action ($O(1)$ duplicate state detection).
3. **Emergency Tripwire:** If the cycle detector determines AURA has entered a loop (e.g., attempting the exact same failed action in the exact same state 3 times), trigger the **Emergency Tripwire Circuit Breaker**, immediately forcing the state to `HALTED` and logging a critical security violation.

---

3. Squadron Integration & Master Test Suite

Combine your four parts into the single `ps06_solution.ipynb` Jupyter Notebook. You must include the following 4 automated test suites (using `assert` statements) at the bottom of your notebook:

1. **State Machine Test:** Assert that the agent transitions correctly from `IDLE` to `HALTED` and strictly terminates if forced to run 21 steps.
2. **Protocol Dispatch Test:** Assert that passing an invalid JSON payload to the dispatcher safely returns a formatted string error observation and does NOT crash the Python process.
3. **MDP Return Test:** Given a hardcoded trajectory with rewards `[10, 0, -5, 20]` and $\gamma = 0.95$, assert the calculated G_0 matches the expected mathematical output.
4. **Sentinel Cycle Test:** Provide the agent with a mock state-action sequence that repeats `[s1, s2, s3, s1, s2, s3...]`. Assert that the Tortoise and Hare algorithm detects the cycle and halts the agent before step 10.

4. Submission Guidelines & Space Crystal Grading Rubric

Compile all roles into `ps06_solution.ipynb`.
Upload the notebook via the USSE Noosphere Command Terminal (Canvas).

Total: 4 Space Crystals

- 💎 **1 Crystal:** `AgentState` enum and step budget enforcement function correctly without external frameworks.
- 💎 **1 Crystal:** Action dispatcher successfully validates tools and wraps errors gracefully.
- 💎 **1 Crystal:** Trajectory logging and G_0 calculation are mathematically accurate.
- 💎 **1 Crystal:** Cycle detection algorithm successfully intercepts and halts infinite loops.

Good luck, Bridge Crew. Eta Carinae is unforgiving. Let's get AURA thinking clearly again.